

Movies Data Manager: An Intelligent Recommendation System Using Custom Data Structures

Areesh Nadeem

BS Data Science

National University of Computer and Emerging Sciences

Islamabad, Pakistan

i242571@isb.nu.edu.pk

Fizza Hussain

BS Data Science

National University of Computer and Emerging Sciences

Islamabad, Pakistan

i242621@isb.nu.edu.pk

Abstract—This project implements a movie management system using custom-built data structures without using the C++ Standard Template Library (STL). In this project, the AVL trees are for sorted movie storage, hash tables are used for quick actor and genre lookups and graphs are used to find relationships between movies. The system processes 4,919 unique movies from the IMDb 5000 dataset from Kaggle and it handles 6,255 unique actors and 26 unique genres. The recommendation engine suggests similar movies based on shared directors (15 points), actors (10 points) and genres (2 points) using a weighted scoring system. The system also has a search system based on multiple categories like movie title, actor name, year, genre and rating range. These operations complete in under 0.5 seconds and the recommendation system generates results in 0.036 seconds. This project shows how data structures can be used to solve real-world problems and create systems that are quick and efficient.

Index Terms—Data Structures, AVL Tree, Hash Table, Graph Algorithms, Linked Lists, BFS, Movie Recommendation System, C++

I. INTRODUCTION

Movie streaming platforms like Netflix and IMDb need fast ways to search movies and recommend new ones to watch. However, the need for such a system is not limited to the entertainment industry, it extends to other industries such as e-commerce, social media and even healthcare, where quick and efficient search and recommendation systems enhance user experience a lot. This project builds a similar system from scratch using basic data structures, without using any pre-made C++ libraries.

We use the IMDb 5000 Movie Dataset, which contains about 5,043 movies with information like movie titles, directors, actors, genres, and ratings. The challenge is to organize this data in such a way that the users can quickly search for movies and get good recommendations.

Our movie data manager system does three main things. First, it lets users search for movies in different ways, by title, actor name, genre, year, or rating. Second, it recommends similar movies based on what they share like same actors, director, or genre. Third, it finds the shortest connection path

between any two movies, showing how they're related through actors or genres.

We chose specific data structures for specific jobs. AVL trees keep movies sorted and enable fast searching. Hash tables provide instant lookups for actors and genres. Graphs show relationships between movies and enable on point recommendations. All of this is built from scratch which gives us complete control and a deep understanding of how everything works behind the scenes.

II. METHODOLOGY

A. System Architecture

The system has four main components working together. The AVL Tree stores all movies sorted by title and year. Hash Tables index actors and genres for quick lookups. The Graph connects related movies that share actors or genres. Linked Lists manage data which doesn't have a fixed length like a movie's actors or genres.

When the system starts, it loads the CSV file and parses each movie. Every movie goes into the AVL tree, and its actors and genres go into hash tables. After loading all movies, the system builds a graph by connecting movies that share attributes. This setup takes about 46 seconds but only happens once. After that, all searches and recommendations are almost instant.

B. Data Structure Implementation

1) *AVL Tree*: Movies are stored in an AVL tree sorted alphabetically by title, with year as a tiebreaker in case there are movies with same title and different year. This handles remakes or movies with the same name. The tree self-balances using rotations (LL, RR, LR, RL) to guarantee $O(\log n)$ search time. We use case-insensitive comparison so searching "avator" or "Avatar" works the same way without throwing any error.

2) *Hash Tables*: We built two hash tables, one for actors and the other one for genres. The hash function uses name length as the key. When multiple names have the same length (collision), we use chaining with linked lists. Each hash entry

Algorithm 1 AVL Tree Rotations

```
RotateRighty
1:  $x \leftarrow y.left$ 
2:  $B \leftarrow x.right$ 
3:  $x.right \leftarrow y$ 
4:  $y.left \leftarrow B$ 
5: Update heights of  $y$  and  $x$ 
6: return  $x$ 
7: RotateLeftx
8:  $y \leftarrow x.right$ 
9:  $B \leftarrow y.left$ 
10:  $y.left \leftarrow x$ 
11:  $x.right \leftarrow B$ 
12: Update heights of  $x$  and  $y$ 
13: return  $y = 0$ 
```

contains a name and a list of movies. This gives us $O(1)$ average lookup time.

Algorithm 2 Hash Table Insertion with Chaining

Require: String *actorName*, MovieNode *movie*

Ensure: Actor-movie pair inserted in hash table

```
1:  $index \leftarrow \text{CalcHash}(actorName)$ 
2:  $current \leftarrow table[index]$ 
3: while  $current \neq \text{NULL}$  do
4:   if  $current.actorName = actorName$  then
5:      $current.AddMovie(movie)$ 
6:     return
7:   end if
8:    $current \leftarrow current.next$ 
9: end while
10: {Actor not found, create new node}
11:  $newActor \leftarrow \text{new ActorNode}(actorName)$ 
12:  $newActor.AddMovie(movie)$ 
13:  $newActor.next \leftarrow table[index]$ 
14:  $table[index] \leftarrow newActor = 0$ 
```

3) *Graph*: Each movie is a graph node. Edges connect movies that share at least one actor or genre. We use adjacency lists to store connections, this means that each movie has a dynamic array of pointers to connected movies. This structure enables the recommendation and path-finding features and make them efficient.

4) *Memory Management*: The AVL tree owns all MovieNode objects. Hash tables and the graph only store pointers to these movies. When the tree is destroyed, it deletes all movies. This clear ownership prevents memory leaks.

C. CSV Parsing

The IMDb dataset has messy data like quoted fields with commas inside, actors separated by the pipe symbol, missing values, and Unicode whitespace characters. Our parser handles these issues by respecting quote boundaries and trimming all whitespace (including Unicode non-breaking spaces like 0xC2 0xA0). We also reset numeric variables to zero at the start of each row to prevent old values from carrying over when data is missing.

III. KEY ALGORITHMS

A. Breadth-First Search for Recommendations

The recommendation engine uses BFS to find similar movies. Starting from a given movie, it searches all directly connected movies and scores them based on shared attributes:

- Same director: 15 points
- Each shared actor: 10 points
- Each shared genre: 2 points
- Similar year (± 3 years): 5 points
- Similar rating (± 1.0): 3 points

After scoring all connections, the system sorts them and returns the top N recommendations. This weighted approach prioritizes strong connections like shared directors over weaker ones like shared genres, producing better recommendations than simple genre matching.

B. Shortest Path Finding

Finding the connection path between two movies also uses BFS. The algorithm explores movies level by level, tracking each movie's parent. When it finds the target movie, it reconstructs the path by backtracking through parents. The path display shows not just the movies but also what connects them which actor, genre, or director.

This runs in $O(V + E)$ time and completes in under 0.01 seconds for our dataset.

IV. RESULTS

A. Dataset Statistics

- Total rows processed: 5,043
- Unique movies: 4,919
- Duplicates: 124
- Unique actors: 6,255
- Unique genres: 26

B. Performance

Initialization:

- CSV loading: 1.5 seconds
- Graph building: 46.3 seconds

Search Operations:

- Title search: 0.021 seconds
- Actor search: 0.022 seconds
- Genre search: 0.117 seconds
- Year search: 0.425 seconds
- Rating range: 0.135 seconds
- Recommendations: 0.036 seconds
- Shortest path: 0.009 seconds

All searches complete in under half a second, making the system feel instant to users. Graph construction takes about 46 seconds because it checks all movie pairs $O(n^2)$, but this only happens once at startup.

Algorithm 3 Find Shortest Path Between Movies (BFS)**Require:** Movie *start*, Movie *end***Ensure:** Shortest path from *start* to *end*

```

1: if start = NULL or end = NULL or start = end then
2:   return
3: end if
4: Initialize visited[movieCount]  $\leftarrow$  false
5: Initialize parent[movieCount]  $\leftarrow$  -1
6: Initialize queue Q
7: startIdx  $\leftarrow$  FindMovieIndex(start)
8: endIdx  $\leftarrow$  FindMovieIndex(end)
9: Enqueue(Q, start)
10: visited[startIdx]  $\leftarrow$  true
11: found  $\leftarrow$  false
12: while  $\neg$ IsEmpty(Q) and  $\neg$ found do
13:   current  $\leftarrow$  Dequeue(Q)
14:   currentIdx  $\leftarrow$  FindMovieIndex(current)
15:   for i  $\leftarrow$  0 to current.connectionsCount - 1 do
16:     neighborIdx  $\leftarrow$  FindMovieIndex(current.connections[i])
17:     if neighborIdx  $\neq$  -1 and  $\neg$ visited[neighborIdx] then
18:       visited[neighborIdx]  $\leftarrow$  true
19:       parent[neighborIdx]  $\leftarrow$  currentIdx
20:       Enqueue(Q, current.connections[i])
21:       if neighborIdx = endIdx then
22:         found  $\leftarrow$  true
23:         break
24:     end if
25:   end if
26: end for
27: end while
28: if found then
29:   Reconstruct path using parent array
30:   Display path with connection details
31: else
32:   print "No path found"
33: end if

```

C. Recommendation Example

Testing with "Interstellar" (2014, directed by Christopher Nolan):

```

Top 5 Recommendations:
1. Inception (2010) - Rating: 8.8 [Score: 22]
   Connection: Shared Genre (Sci-Fi, Adventure)
   Same Director (Christopher Nolan)
2. The Little Prince (2015) [Score: 22]
   Connection: Shared Actor (Mackenzie Foy)
   Shared Genre (Drama, Adventure)
3. The Prestige (2006) [Score: 22]
   Connection: Shared Genre (Sci-Fi, Drama)
   Same Director (Christopher Nolan)
4. Batman Begins (2005) [Score: 20]
   Connection: Shared Genre (Adventure)
   Same Director (Christopher Nolan)
5. Dallas Buyers Club (2013) [Score: 20]
   Connection: Shared Actor (Matthew McConaughey)
   Shared Genre (Drama)

```

The recommendations make sense because three are other Christopher Nolan films, one shares the lead actor and one shares a supporting actress. This provides variety while staying relevant.

D. Shortest Path Example

Path from Inception to The Dark Knight:
Distance: 1 connection

```

Inception (2010)
|
[Connected via: Genre: Thriller,
 Director: Christopher Nolan]
v
The Dark Knight (2008)

```

The movies connect directly through their shared director and genre.

V. DISCUSSION**A. Major Challenges**

CSV Data Issues: The dataset had invisible Unicode spaces, missing values, and inconsistent formatting. We fixed this with trimming that removes both regular and Unicode whitespace.

Year = 0 Bug: There were movies with empty/missing year field in the CSV file, so instead of discarding those movie entries, we stored 0 internally to avoid errors. In displaying, 0 is shown as 'N/A'. For example, 'Dekalog' has 0 year.

Duplicates: In the dataset, there were movies which had the same title but completely different genres, rating, year and actors. Since AVL Trees don't take duplicate values, it would reject inserting the second movie. However, we wanted to include them, so we added year as another parameter to deal with duplicates. Therefore first our movie count was 4,917 and after applying these changes it became 4,919.

Memory Management: Sharing MovieNode pointers across multiple structures was tricky. We made the AVL tree the owner of all movies, while other structures only hold pointers.

Recommendation Quality: Initial equal weighting produced generic genre-based recommendations. After tuning weights to prioritize directors (15) and actors (10) over genres (2), recommendations improved significantly.

String Comparison: Case sensitivity and extra spaces caused search misses. We implemented case-insensitive comparison and trimming.

B. Limitations

The $O(n^2)$ graph construction doesn't scale well beyond 10,000 movies. For larger datasets, better algorithms or approximations would be needed.

The hash function based on name length creates longer chains than necessary. A better hash function using character values would improve distribution.

Recommendations only consider direct connections. Exploring movies at distance 2 or 3 could find more interesting suggestions.

C. Learning Outcomes

Building data structures from scratch taught us how they really work. We learned why AVL trees need balancing, how hash collisions happen, and how BFS explores graphs efficiently.

Manual memory management forced us to think about ownership and lifetime for every pointer.

The recommendation tuning showed that good algorithms need rigorous testing. Our first design looked reasonable to us at first but after testing and analyzing the results, we realized that it needed adjustment.

VI. CONCLUSION

This project successfully demonstrates how data structures solve real world problems. We built a complete movie management system with fast search, recommendations and a shortest path finding.

The system handles nearly 5,000 movies efficiently, with all operations completing in under half a second. The weighted recommendation system produces relevant suggestions by prioritizing strong connections like shared directors and actors.

Building everything from scratch gave us deep insights into how data structures work internally. The challenges we faced, from messy CSV data to memory management complexities, taught us valuable debugging and problem-solving skills.

While the system has limitations for very large datasets, it successfully achieves all project requirements and demonstrates the power of choosing the right data structure for each task. The experience of implementing AVL trees, hash tables, and graphs manually gave us understanding that using pre-built libraries would have hidden.

ACKNOWLEDGMENTS

A huge thank you to our course instructor, Ma'am Mahnoor Tariq and our lab instructor, Sir Abdul Wahab, for all their hard work and support throughout the course. We're grateful for their efforts and the way they always made sure that we understood the concepts.